Enterprise Role-Aware AI Assistant for Learning Management System

Technical Manual for Chatbot System

By Jatin Verma
Organization: Reliance Jio Platforms Ltd.
Project: Enterprise LMS Chatbot Assistant
Version v2: Review Documentation and Manual

GITHUB LINK: https://github.com/jatinverma2004/ChatbotLMSCodes

Table of Contents

1. Executive Summary

2. Project Objectives

3. Problem Context

4. System Overview

5. Overall Architecture

6. Technology Stack

7. Repository Structure

8. MCP Context Server

9. Database and Data Model

10. Chatbot Service Layer

11. LangGraph Orchestration

12. Metadata-Driven SOP Ranking

13. Semantic Retrieval Layer

14. User Interface (Chat UI)

15. End-to-End System Workflow

16. Deployment & Execution

17. Codebase Structure and File Responsibilities

1. Executive Summary

This project implements a role-aware AI assistant designed to support enterprise employees in accessing Standard Operating Procedures (SOPs) stored within a Learning Management System (LMS).

The system enables employees to ask natural language questions and receive contextual answers derived from relevant SOP documents associated with their organizational role.

The architecture integrates a Model Context Protocol (MCP) server, a chatbot service layer, and a local Large Language Model (LLM).

The system emphasizes secure enterprise deployment by running all AI inference locally while enforcing role-based document access through structured metadata.

2. Project Objectives

The primary objective of the system is to simplify enterprise knowledge access by enabling employees to interact with SOP documentation conversationally.

Key goals of the project include:

- Provide role-aware document retrieval based on employee designation.
- Reduce manual search across large document repositories.
- Deliver contextual AI responses grounded in enterprise SOPs.
- Maintain enterprise data privacy by using local model inference.
- Implement scalable architecture suitable for future enterprise deployment.

3. Problem Context

Large organizations maintain extensive libraries of Standard Operating Procedures distributed across departments and functions.

Employees often face challenges identifying which documents apply to their specific roles and responsibilities.

Traditional LMS platforms provide static document storage but lack intelligent mechanisms to interpret employee queries and retrieve relevant procedural knowledge.

The proposed AI assistant addresses this gap by combining role context, metadata filtering, and semantic search techniques to deliver accurate responses.

4. System Overview

The AI assistant consists of three primary layers:

1. Context Layer – MCP server responsible for managing enterprise metadata and user profiles.

2. Intelligence Layer – Chatbot service integrating LangGraph pipelines and LLM inference.

3. Interaction Layer – Streamlit based chat interface used by employees.

The system retrieves user context, identifies relevant SOP documents, extracts relevant knowledge segments, and generates responses using the LLM.

5. Overall Architecture

The architecture follows a modular microservice design to maintain separation of responsibilities between services.

Major components include:

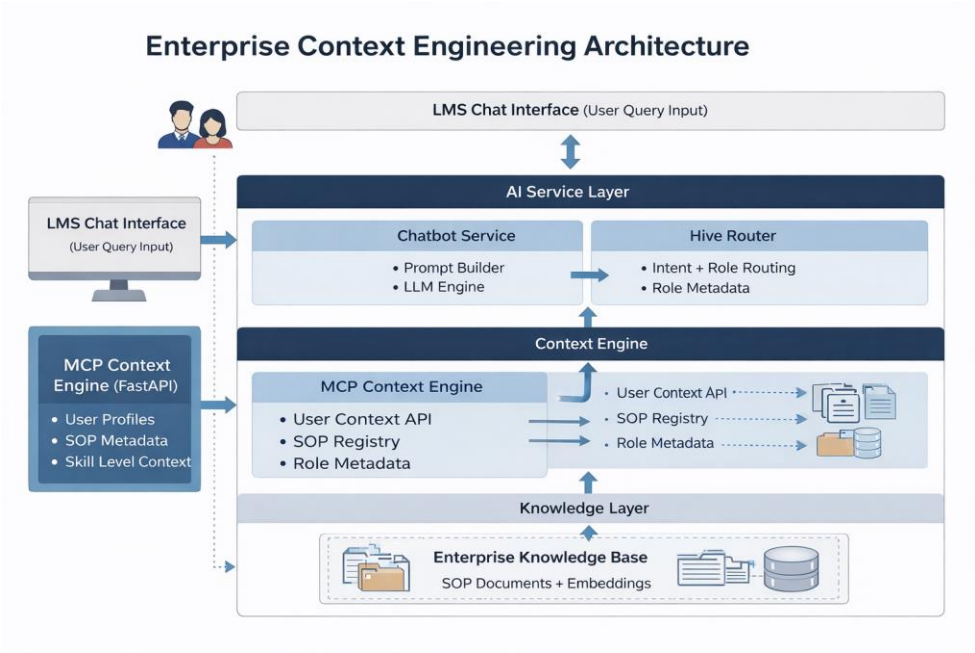
• MCP Context Server – maintains organizational data and SOP metadata.

• Chatbot Service – processes user queries and manages AI reasoning workflow.

• LangGraph Execution Engine – orchestrates document retrieval and response generation.

• Local LLM (Ollama / Phi-3) – generates responses based on retrieved SOP context.

• Streamlit Chat Interface – provides employee interaction layer.



6. Technology Stack

Programming Language: Python

Backend Framework: FastAPI

AI Orchestration: LangGraph

LLM Runtime: Ollama (Phi-3 local model)

Semantic Embeddings: Sentence-Transformers

Frontend Interface: Streamlit

Database: SQLite

Document Parsing: PyPDF, python-docx

Version Control: GitHub

7. Repository Structure

The system repository is organized into modular service directories.

Key folders include:

• mcp_v2_server – earlier version of the MCP context engine.

• mcp_v3_server – enhanced version including metadata and embedding capabilities.

• chatbot_service – contains chatbot logic, LangGraph workflow, and chat UI.

This separation allows independent development and testing of context services and chatbot functionality.

8. MCP Context Server

The MCP server acts as the enterprise context layer between the chatbot and organizational knowledge.

It exposes APIs that allow the chatbot to retrieve information about employees, job roles, and associated SOP documents.

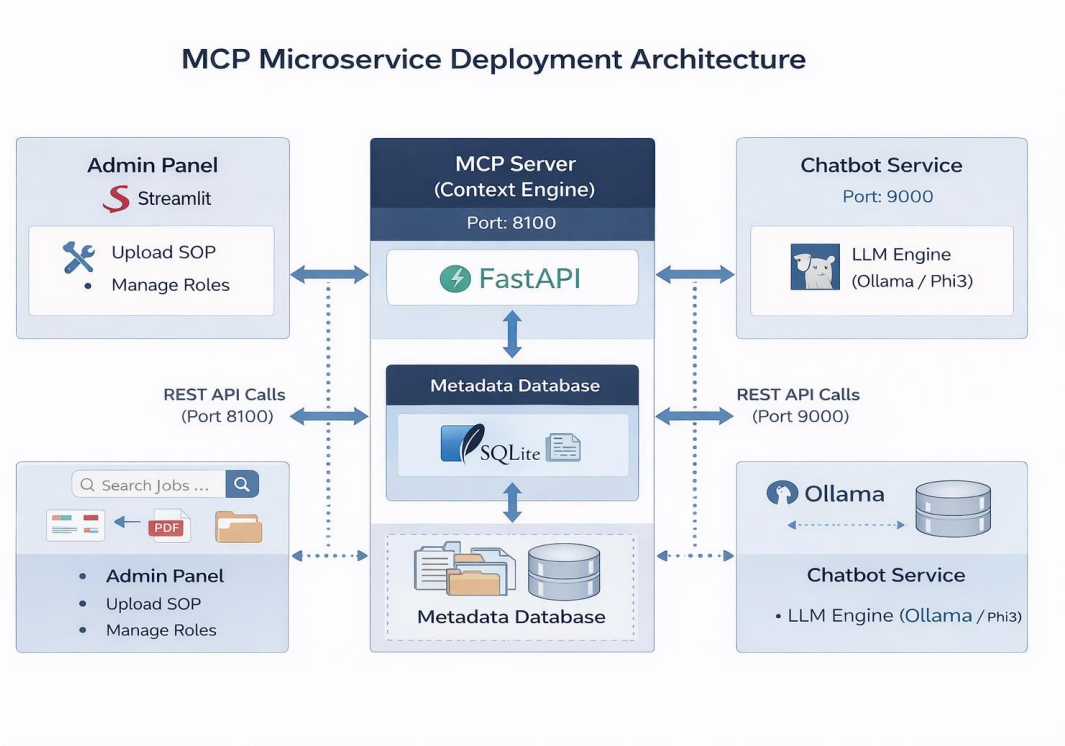
Core responsibilities include:

• Managing employee profiles and job roles.

• Maintaining SOP registry and document metadata.

• Providing APIs for retrieving role-specific SOP lists.

• Supporting metadata filtering for improved document selection.



9. Database and Data Model

The system uses a lightweight SQLite database for storing enterprise metadata.

Key tables include:

• user_profiles – employee identifiers and role information.

• sop_registry – records of SOP documents and their metadata.

• job_role_skills – mapping between roles and required skills.

• skills_registry – master list of organizational skills.

• user_skills_map – mapping of employees to skill levels.

• sop_metadata – additional attributes used for ranking and prioritization.

10. Chatbot Service Layer

The chatbot service acts as the central processing unit for conversational interactions.

It receives queries from the UI, retrieves relevant context from the MCP server, and orchestrates the LangGraph pipeline.

Major tasks performed by the chatbot service include:

• Query interpretation and routing.

• Retrieval of role-specific SOP metadata.

• Semantic matching between queries and SOP documents.

• Construction of prompts for the language model.

• Delivery of final responses to the user interface.

11. LangGraph Orchestration

LangGraph is used to structure the reasoning workflow of the AI assistant.

The workflow pipeline consists of multiple nodes including:

1. Fetch user role context.

2. Retrieve mapped SOP documents.

3. Identify the best matching SOP using metadata and semantic similarity.

4. Load relevant document content.

5. Generate response using the local LLM.

12. Metadata-Driven SOP Ranking

Metadata was introduced to improve document prioritization and ensure the chatbot retrieves the most relevant SOP.

Each SOP is associated with attributes such as:

• Skill Level (S1, S2, S3)

• Criticality Level (LOW, MEDIUM, HIGH)

• Proficiency Requirement

• Last Updated Timestamp

These attributes allow the system to rank documents and select the most relevant SOP when multiple matches exist.

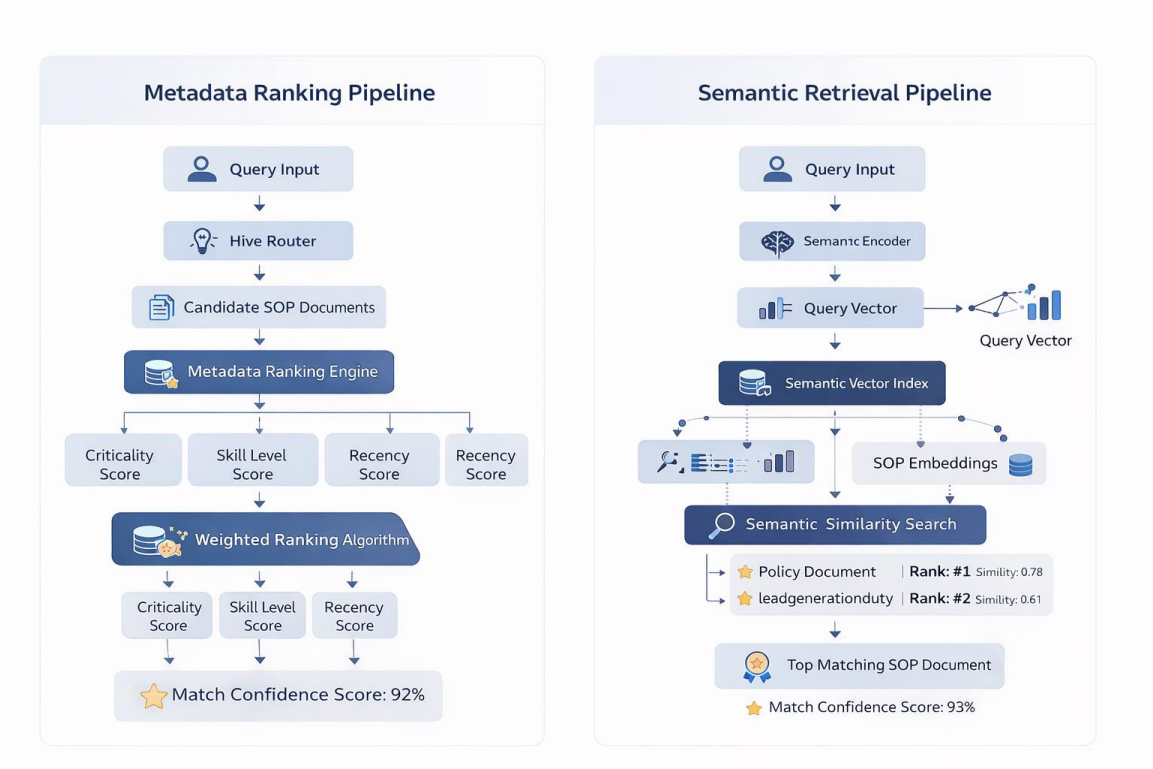
13. Semantic Retrieval Layer

In addition to metadata filtering, the system incorporates semantic retrieval using vector embeddings.

SOP titles are converted into numerical vector representations using the Sentence-Transformers model.

When a user submits a query, the system generates a vector for the query and calculates similarity scores against SOP embeddings.

This approach enables meaning-based matching rather than simple keyword matching, improving retrieval accuracy.



14. User Interface (Chat UI)

The user interface is implemented using Streamlit.

It provides a lightweight web interface allowing employees to interact with the AI assistant.

Key features include:

• Employee login using employee ID.

• Display of role-specific SOP documents in a sidebar.

• Interactive chat interface for asking questions.

• Display of generated responses from the AI assistant.

15. End-to-End System Workflow

The complete system workflow follows these steps:

1. User logs into the interface using employee ID.

2. The system retrieves role context from the MCP server.

3. The employee submits a natural language question.

4. Chatbot service determines the relevant SOP using metadata and semantic similarity.

5. SOP content is retrieved from the document repository.

6. Context and query are combined to form an LLM prompt.

7. The local LLM generates the final response.

8. The response is displayed in the chat interface.

16. Deployment & Execution

The system is deployed locally using Python virtual environments.

Multiple services run independently on separate ports:

• MCP Server

• Chatbot Service

• LangGraph Engine

• Streamlit UI

Each component communicates via REST APIs to maintain modular architecture.

17. Codebase Structure and File Responsibilities

MCP_V2_SERVER

File Name	Description
main.py	Implements the initial version of the MCP (Model Context Protocol) server using FastAPI. It provides APIs for retrieving employee context, SOP information, and document access. This version focuses on basic role-to-SOP mapping without advanced metadata ranking.
init_db.py	Initializes the SQLite database schema used by the MCP server. It creates tables required to store employee profiles and basic SOP registry information.
generate_embeddings.py	Generates vector embeddings for SOP document names using the SentenceTransformers model. These embeddings are stored for semantic similarity matching during query processing.
populate_metadata.py	Populates metadata fields such as criticality, proficiency level, and timestamps for SOP records. This script helps enrich the database with ranking attributes used by later retrieval mechanisms.
mcp_v3.db	SQLite database storing structured enterprise metadata including employee profiles, SOP registry, skill mappings, and document attributes.

MCP_V3_SERVER

File Name	Description
main.py	Core MCP server implementation using FastAPI. It exposes REST APIs for managing employee context, SOP registry, skill mappings, metadata ranking, and document retrieval. This version introduces advanced metadata-driven SOP prioritization.
init_db.py	Responsible for creating and initializing database tables required by MCP V3. It ensures required schema elements such as user profiles, SOP registry, and skill mapping tables exist.

File Name	Description
populate_metadata.py	Automatically assigns metadata attributes to SOP records based on skill levels and criticality. This metadata improves SOP ranking during query matching.
generate_embeddings.py	Creates semantic vector representations for SOP titles using the SentenceTransformers embedding model. These embeddings enable meaning-based document matching rather than simple keyword matching.
sop_storage/	Directory that stores uploaded SOP documents such as PDF and DOCX files. These documents are accessed dynamically by the chatbot service during response generation.
mcp_v3.db	Primary SQLite database storing user profiles, SOP registry entries, metadata attributes, and role-skill mappings used by the chatbot system.

CHATBOT_SERVICE

File Name	Description
app.py	Implements the main chatbot API service using FastAPI. It handles incoming user queries, retrieves role context from the MCP server, and orchestrates response generation using SOP content and the local LLM.
chat_ui.py	Streamlit-based web interface that allows employees to interact with the chatbot. It manages employee login, displays role-specific SOP documents, and presents conversational responses.
langgraph_engine.py	Defines the LangGraph workflow used to orchestrate the AI reasoning pipeline. It connects different nodes responsible for context retrieval, SOP matching, and response generation.
langgraph_chat.py	FastAPI wrapper that exposes the LangGraph pipeline as an API endpoint. It receives user messages, executes the LangGraph workflow, and returns chatbot responses.
graph_nodes.py	Contains individual processing nodes used in the LangGraph pipeline. These nodes perform tasks such as retrieving employee context, matching SOPs, loading document text, and invoking the

File Name	Description
	LLM.
graph_state.py	Defines the shared state structure passed between LangGraph nodes. It stores information such as user context, matched SOP documents, and generated responses.
ollama_client.py	Handles communication with the Ollama local LLM runtime. It sends prompts to the model and retrieves generated responses used by the chatbot.
hive_router.py	Implements a deterministic routing mechanism that maps user queries to specific SOP categories based on keyword detection. This reduces ambiguity during document selection.
system_prompt.py	Stores the base system prompt template used for LLM interactions. It defines conversational tone, response structure, and enterprise behavior rules.
dev_runner.py	Utility script used during development to start multiple services simultaneously, including MCP server, chatbot API, LangGraph service, and Streamlit UI.